Unit III Searching and Sorting

Prof. Pranjal Pandit

Searching and Sorting

Searching:

- sequential/linear search,
- sentinel search,
- binary search,
- Fibonacci search,
- indexed sequential search.

Sorting:

- internal vs external sorting;
- sort order, stability, efficiency and number of passes;
- comparison-based methods (Bubble, Insertion, Selection, Quick, Shell);
- non-comparison-based methods (Radix, Counting, Bucket);
- comparison of all methods and their complexities.

Sorting

- Sorting is the process of arranging data in a particular order (usually ascending or descending) so that searching, merging, and data processing become easier and faster.

Example

Unsorted Array:

45 12 89 5 33

Ascending Order:

5 12 33 45 89

Descending Order:

89 45 33 12 5

Internal Sorting Example

Suppose an array contains

50 20 80 10 40

Since all numbers fit in memory, Quick Sort or Bubble Sort can be used.

Internal Sorting vs External Sorting

Internal Sorting	External Sorting
Entire data fits into RAM.	Data is too large to fit into RAM.
Faster because data is in memory.	Slower because disk access is required.
Used for small to medium datasets.	Used for huge datasets (GBs or TBs).
Examples: Bubble, Insertion, Quick, Merge, Heap	Examples: External Merge Sort

External Sorting Example

Suppose a bank has 1 Billion customer records These cannot fit into RAM.

Steps:

- Divide data into smaller chunks.
- Sort each chunk in memory.
- Store sorted chunks on disk.
- Merge all sorted chunks.

This process is called **External Merge Sort**.

Sort Order

Sorting can be performed in two ways.

(a) Ascending Order

Smallest to largest.

(b) Descending Order

Largest to smallest.

Stability of Sorting

A sorting algorithm is **stable** if equal **elements keep their original relative order** after sorting.

Example

Student Records. Sort according to Marks.

Roll	Name	Marks
101	Amit	80
102	Rahul	70
103	Riya	80
104	Neha	60

Stable Sort

60 Neha

70 Rahul

80 Amit

80 Riya

Amit remains before Riya.

Unstable Sort

60 Neha

70 Rahul

80 Riya

80 Amit

Relative order changed.

Stable Algorithms

- Bubble Sort
- Insertion Sort
- Merge Sort
- Counting Sort
- Radix Sort

Unstable Algorithms

- Selection Sort
- Quick Sort
- Heap Sort
- Shell Sort

Efficiency:

Efficiency measures the performance of an algorithm. It is measured using

- Time Complexity
- Space Complexity

Example

Suppose n = 1000

Bubble Sort performs nearly

$$1000^2 = 1{,}000{,}000 \text{ comparisons}$$

Quick Sort performs approximately

$$1000 \log_2 1000 \approx 1000 \times 10 \approx 10{,}000 \text{ comparisons}$$

Hence Quick Sort is much faster.

Number of Passes

A **pass** means one complete traversal over the array.

Example

5 4 3 2 1

Bubble Sort needs

$n-1 = 5-1$

$= 4$ passes

Comparison-Based Sorting Algorithms

These algorithms compare two elements before deciding their order.

Examples:

- Bubble Sort
- Insertion Sort
- Selection Sort
- Quick Sort
- Shell Sort
- Merge Sort
- Heap Sort

Bubble Sort

Principle

- Repeatedly compare adjacent elements.
- If they are in the wrong order, swap them.
- The largest element "bubbles" to the end after every pass.

Algorithm

```
BubbleSort(A,n)

for i=0 to n-2

    for j=0 to n-i-2

        if A[j] > A[j+1]

            swap(A[j],A[j+1])
```

Example

64 34 25 12 22

Pass 1

64 34

Swap

34 64 25 12 22

64 25

Swap

34 25 64 12 22

64 12

Swap

34 25 12 64 22

64 22

Swap

34 25 12 22 64

Largest element reached the end

Pass 2

34 25

Swap

25 34 12 22 64

34 12

Swap

25 12 34 22 64

34 22

Swap

25 12 22 34 64

Pass 3

25 12

Swap

12 25 22 34 64

25 22

Swap

12 22 25 34 64

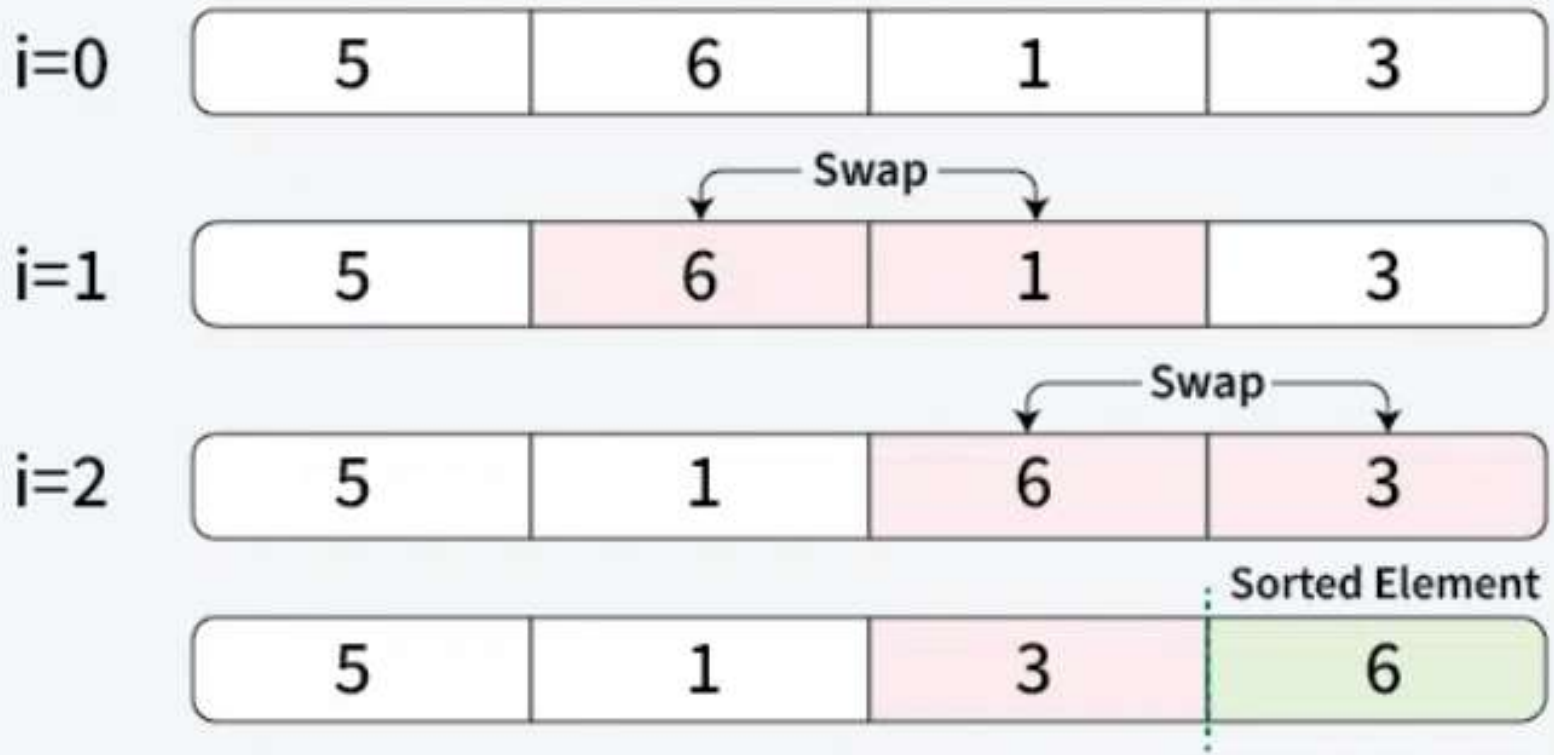
Pass 4

Already Sorted

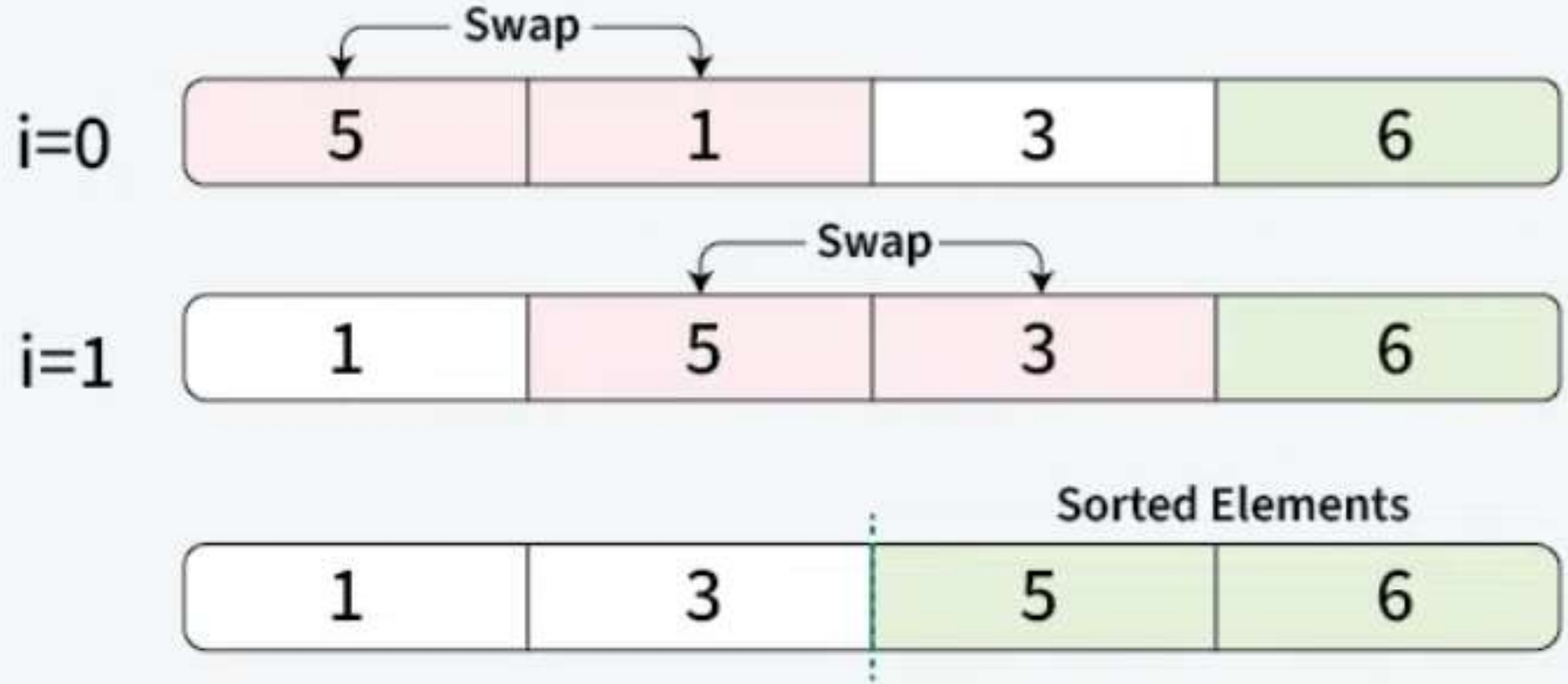
12 22 25 34 64

01 Step

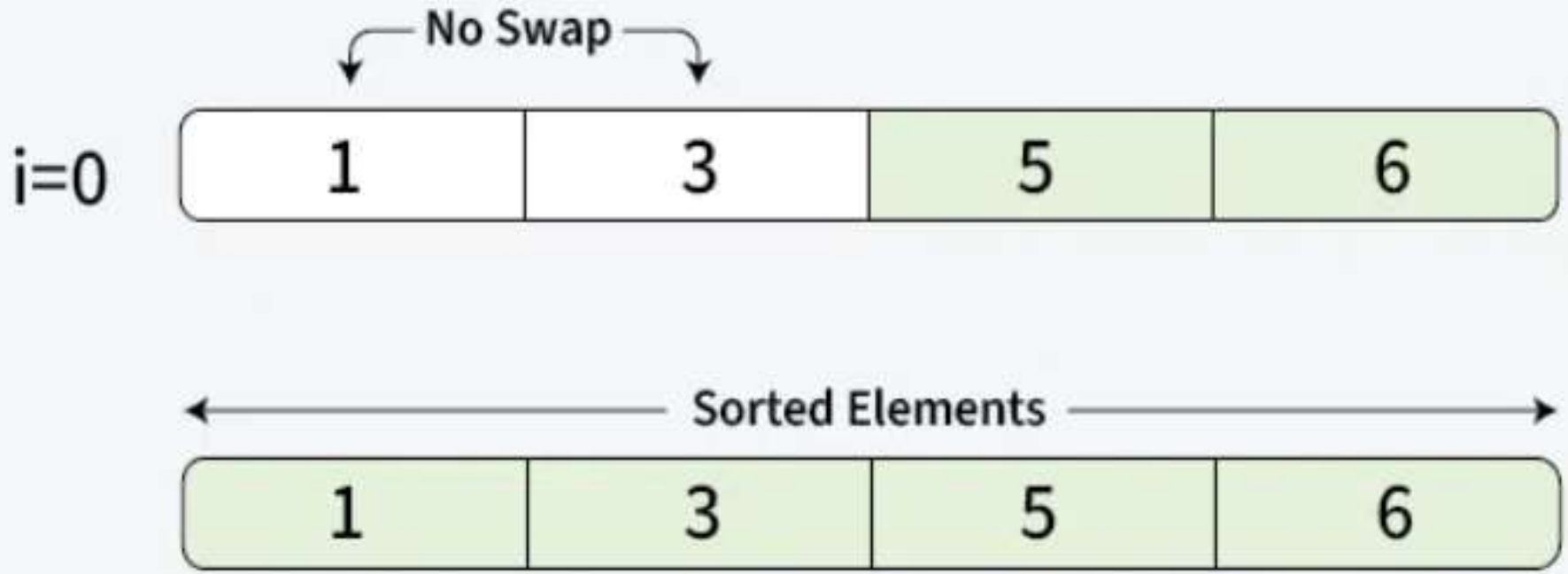
Placing the 1st largest element at its correct position



02 Step | Placing 2nd largest element at its correct position



03 Step | Placing 3rd largest element at its correct position



```
#include <iostream>
using namespace std;
int main()
{
   int arr[] = {64,34,25,12,22};
   int n = 5;
   for(int i=0;i<n-1;i++)
   {
      for(int j=0;j<n-i-1;j++)
      {
         if(arr[j]>arr[j+1])
            swap(arr[j],arr[j+1]);
      }
   }
   cout<<"Sorted Array:\n";
   for(int i=0;i<n;i++)
      cout<<arr[i]<<" ";
}
```

Complexity

Space = O(1)

Stable = Yes

Case	Time
Best	O(n) (optimized version with swap flag)
Average	$O(n^2)$
Worst	$O(n^2)$

It can be optimized by stopping the algorithm if the inner loop didn't cause any swap.

Insertion Sort

Principle

- Just like arranging playing cards.
- Take one element and insert it into the correct position.
- The Insertion Sort algorithm uses one part of the array to hold the sorted values, and the other part of the array to hold values that are not sorted yet.

Example

12 11 13 5 6

Insert 11

11 12 13 5 6

Insert 13

11 12 13 5 6

Insert 5

5 11 12 13 6

Insert 6

5 6 11 12 13

Manual Run Through

Before we implement the Insertion Sort algorithm in a programming language, let's manually run through a short array, just to get the idea.

Step 1: We start with an unsorted array.

```
[ 7, 12, 9, 11, 3]
```

Step 2: We can consider the first value as the initial sorted part of the array. If it is just one value, it must be sorted, right?

```
[ 7, 12, 9, 11, 3]
```

Step 3: The next value 12 should now be moved into the correct position in the sorted part of the array. But 12 is higher than 7, so it is already in the correct position.

```
[ 7, 12, 9, 11, 3]
```

Step 4: Consider the next value 9.

```
[ 7, 12, 9, 11, 3]
```

Step 5: The value 9 must now be moved into the correct position inside the sorted part of the array, so we move 9 in between 7 and 12.

```
[ 7, 9, 12, 11, 3]
```

Step 6: The next value is 11.

```
[ 7, 9, 12, > 11, 3]
```

Step 7: We move it in between 9 and 12 in the sorted part of the array.

```
[ 7, 9, 11, 12, 3]
```

Step 8: The last value to insert into the correct position is 3.

```
[ 7, 9, 11, 12, 3]
```

Step 9: We insert 3 in front of all other values because it is the lowest value.

```
[ 3,7, 9, 11, 12]
```

Finally, the array is sorted.

Understanding…

- The first value is considered to be the initial sorted part of the array.
- Every value after the first value must be compared to the values in the sorted part of the algorithm so that it can be inserted into the correct position.
- The Insertion Sort Algorithm must run through the array 4 times, to sort the array of 5 values because we do not have to sort the first value.
- And each time the algorithm runs through the array, the remaining unsorted part of the array becomes shorter.
- We will now use what we have learned to implement the Insertion Sort algorithm in a programming language.

```
my_array = [64, 34, 25, 12, 22, 11, 90, 5]

n = len(my_array)
for i in range(1,n):
    insert_index = i
    current_value = my_array.pop(i)
    for j in range(i-1, -1, -1):
        if my_array[j] > current_value:
            insert_index = j
    my_array.insert(insert_index, current_value)

print("Sorted array:", my_array)
```

```cpp
#include<iostream>
using namespace std;
void insertionSort(int *array, int size){
    int key, j;
    for(int i = 1; i<size; i++) {
        key = array[i];//take value
        j = i;
        while(j > 0 && array[j-1]>key) {
            array[j] = array[j-1];
            j--;
        }
        array[j] = key; //insert in right place
    }
}
```

```
int main(){
    int n;
    n = 5;
    int arr[5] = {67, 44, 82, 17, 20}; // initialize the array
    cout << "Array before Sorting: ";
    for(int i = 0; i<n; i++)
        cout << arr[i] << " ";
    cout << endl;
    insertionSort(arr, n);
    cout << "Array after Sorting: ";
    for(int i = 0; i<n; i++)
        cout << arr[i] << " ";
    cout << endl;
}
```

Selection Sort

- This sorting algorithm, like insertion sort, is an in-place comparison-based algorithm in which the list is divided into two parts, the sorted part at the left end and the unsorted part at the right end.

- Initially, the sorted part is empty and the unsorted part is the entire list.

- The smallest element is selected from the unsorted array and swapped with the leftmost element, and that element becomes a part of the sorted array.

- This process continues moving unsorted array boundaries by one element to the right.

- This algorithm is not suitable for large data sets as its average and worst case complexities are of $\mathbf{O(n^2)}$, where **n** is the number of items.

- This type of sorting is called Selection Sort as it works by repeatedly sorting elements.

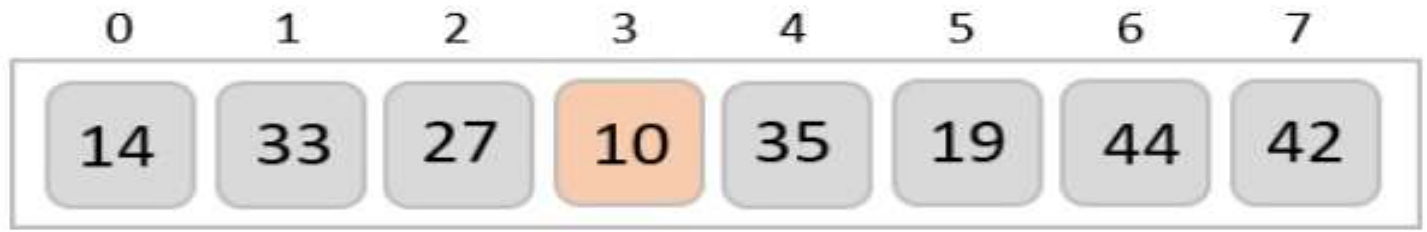
- That is: we first find the smallest value in the array and exchange it with the element in the first position, then find the second smallest element and exchange it with the element in the second position, and we continue the process in this way until the entire array is sorted.

Example

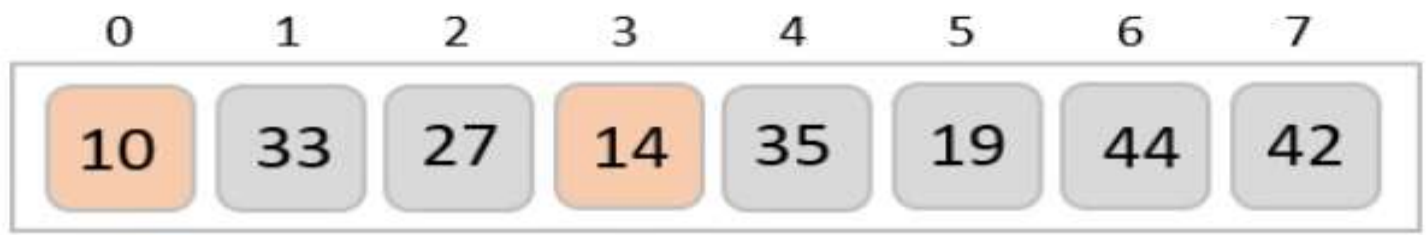
Consider the following depicted array as an example.



For the first position in the sorted list, the whole list is scanned sequentially. The first position where 14 is stored presently, we search the whole list and find that 10 is the lowest value.

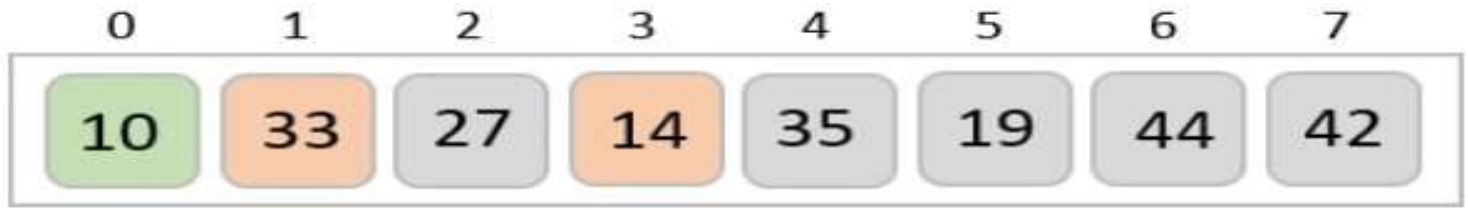


So we replace 14 with 10. After one iteration 10, which happens to be the minimum value in the list, appears in the first position of the sorted list.

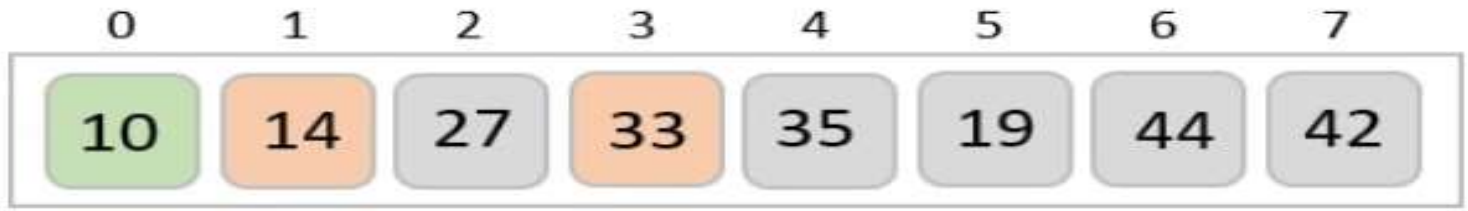


For the second position, where 33 is residing, we start scanning the rest of the list in a linear manner.

For the second position, where 33 is residing, we start scanning the rest of the list in a linear manner.



We find that 14 is the second lowest value in the list and it should appear at the second place. We swap these values.

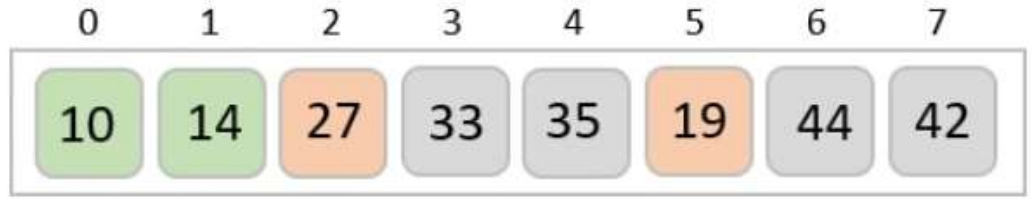


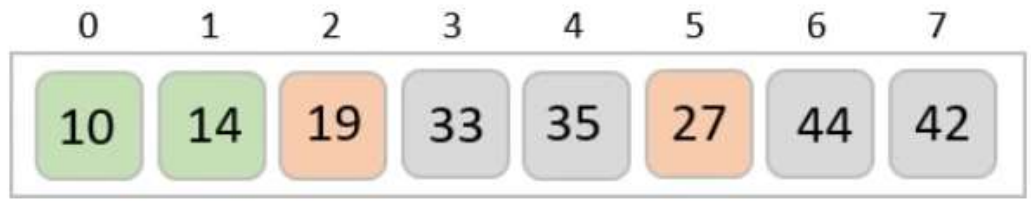
After two iterations, two least values are positioned at the beginning in a sorted manner.

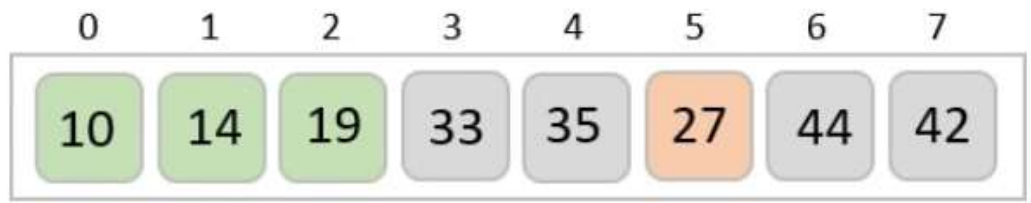


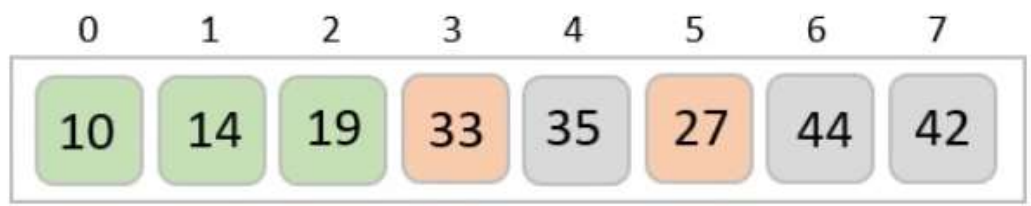
The same process is applied to the rest of the items in the array −

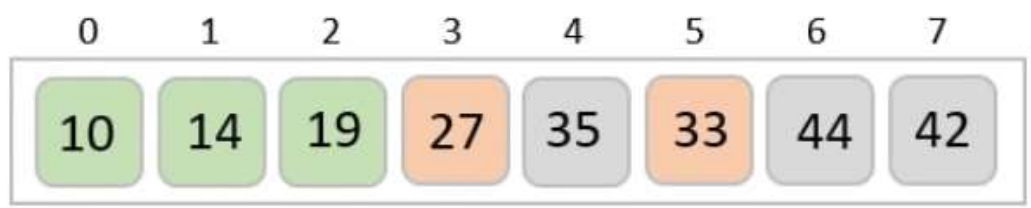


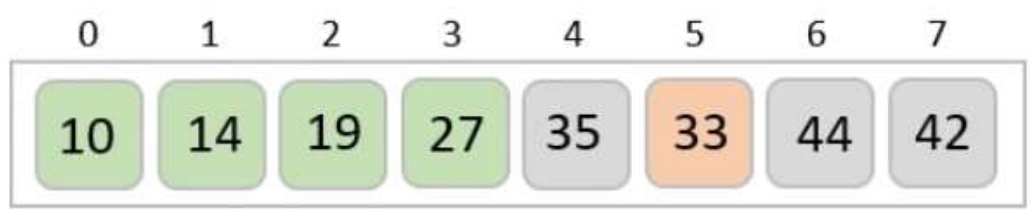

0 1 2 3 4 5 6 7
10 14 27 33 35 19 44 42

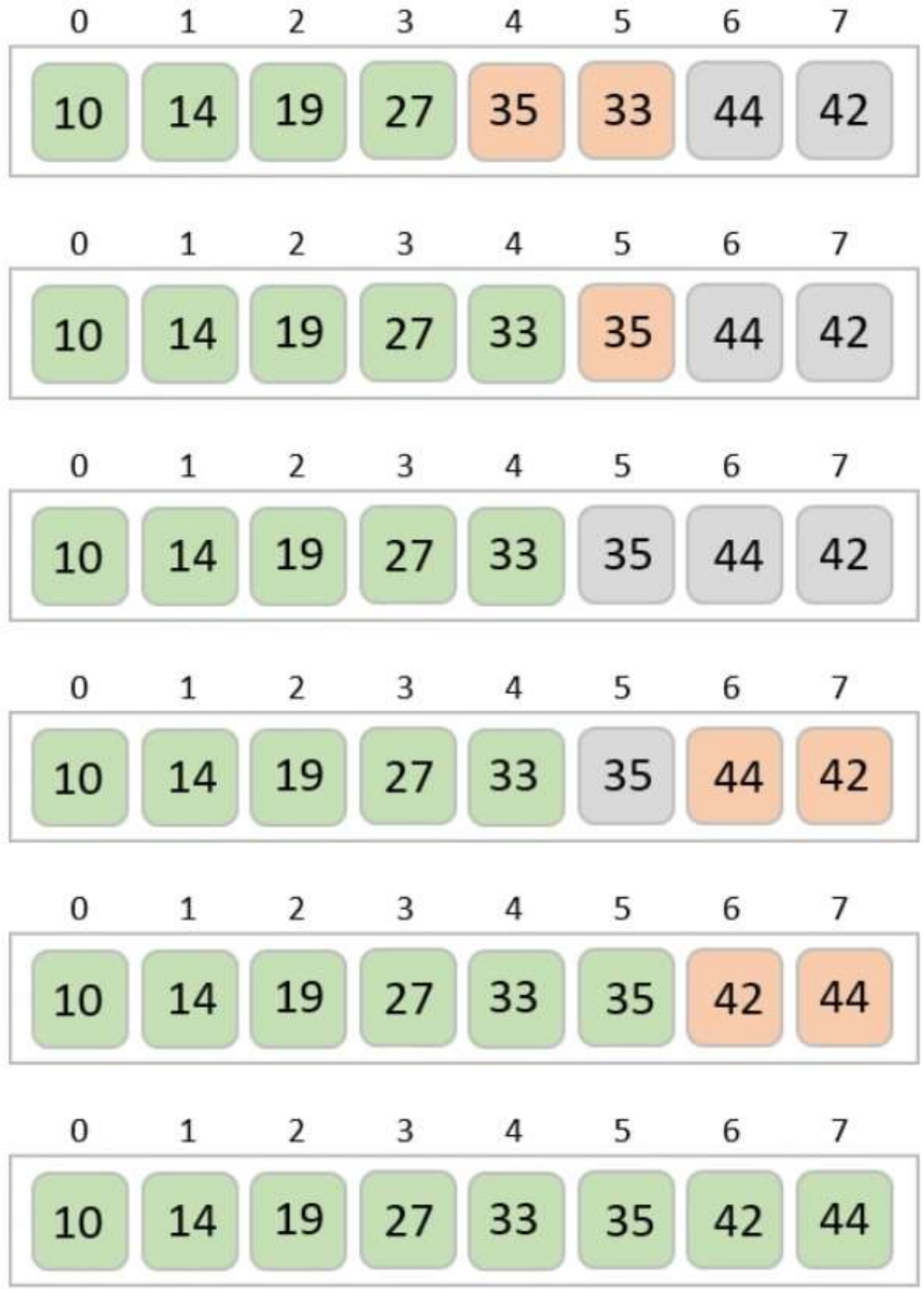

0 1 2 3 4 5 6 7
10 14 19 33 35 27 44 42


0 1 2 3 4 5 6 7
10 14 19 33 35 27 44 42


0 1 2 3 4 5 6 7
10 14 19 33 35 27 44 42


0 1 2 3 4 5 6 7
10 14 19 27 35 33 44 42


0 1 2 3 4 5 6 7
10 14 19 27 35 33 44 42


0 1 2 3 4 5 6 7
10 14 19 27 35 33 44 42
0 1 2 3 4 5 6 7
10 14 19 27 33 35 44 42
0 1 2 3 4 5 6 7
10 14 19 27 33 35 44 42
0 1 2 3 4 5 6 7
10 14 19 27 33 35 44 42
0 1 2 3 4 5 6 7
10 14 19 27 33 35 42 44
0 1 2 3 4 5 6 7
10 14 19 27 33 35 42 44

```
#include<iostream>
using namespace std;
void swapping(int &a, int &b) {  //swap the content of a and b
   int temp;
   temp = a;
   a = b;
   b = temp;
}
void selectionSort(int *array, int size){
   int i, j, imin;
   for(i = 0; i<size-1; i++) {
      imin = i; //get index of minimum data
      for(j = i+1; j<size; j++)
         if(array[j] < array[imin])
            imin = j;
```

```
        //placing in correct position
        swap(array[i], array[imin]);
    }
}
int main(){
    int n;
    n = 5;
    int arr[5] = {12, 19, 55, 2, 16}; // initialize the array
    cout << "Array before Sorting: ";
    for(int i = 0; i<n; i++)
        cout << arr[i] << " ";
    cout << endl;
    selectionSort(arr, n);
    cout << "Array after Sorting: ";
    for(int i = 0; i<n; i++)
        cout << arr[i] << " ";
    cout << endl;
}
```